

CE 488/5588: Applied AI

Topic 15: Introduction to Large Language
Models — Language, Representation, and
Retrieval for Engineering

Lecture Roadmap

- 1 Why This Topic Matters for Engineers
- 2 Why NLP Was Historically Difficult
- 3 Why Modern LLMs Succeeded
- 4 From Text to Tokens
- 5 From Tokens to Vectors
- 6 Vector Stores and Retrieval
- 7 Generation Limits: Hallucination and Reliability
- 8 Conclusion and Transition

A New Part of the Course

Earlier topics focused on machine learning and deep learning as predictive or generative tools.

Topic 15 begins a new part of the course:

- language-centered foundation models,
- document-heavy engineering workflows,
- and the question of how language becomes a computational interface.

This chapter is not mainly about chatbots. It is about why LLMs matter in technical work.

Why Engineers Should Care

Much of engineering work is organized around language-like artifacts:

- design codes and specifications,
- reports and spreadsheets,
- tables, notes, and emails,
- source code and analysis scripts.

Main Point

The first major impact of LLMs on engineering is likely not full automation. It is accelerated interaction with information.

What an LLM-Based Workflow Can Support

A well-designed workflow can help an engineer:

- search large bodies of technical documents in natural language,
- summarize long reports or standards before detailed review,
- compare assumptions across multiple sources,
- draft code, scripts, and notebooks,
- organize planning and research tasks.

The practical benefit is time compression between a question and the relevant technical context.

Language as Engineering Infrastructure

When people hear “language model,” they often think first of a chatbot. For engineering, that view is too narrow.

Broader Interpretation

Language is the surface layer of many engineering systems: code clauses, inspection logs, design memos, spreadsheet notes, and source code are all part of the same information environment.

An LLM can therefore be viewed as a model for interacting with the information layer of engineering work.

Bridge Assessment as a Motivating Example

Bridge Rehabilitation Workflow

An engineer may need to combine inspection notes, prior load-rating reports, AASHTO code clauses, spreadsheets of section properties, emails about field changes, and Python analysis scripts.

None of these sources alone is “the design,” but together they define the knowledge environment in which design decisions are made.

LLM-based systems become useful when they help navigate this environment faster and more coherently.

Learning Objectives

After this chapter, students should be able to:

- explain why language models matter in engineering workflows,
- summarize the main stages of NLP history,
- explain why modern LLMs succeeded,
- describe tokenization, embeddings, and high-dimensional representation,
- explain how vector retrieval returns human-readable text,
- discuss hallucination and reliability in engineering use.

Why Language Was Always Hard for AI

Natural language processing has always been difficult because language is:

- ambiguous,
- context-dependent,
- combinatorial,
- and often domain-specific.

For engineers, the challenge is even harder because technical language mixes prose, equations, abbreviations, units, tables, and references to standards.

Rule-Based Beginnings

Early NLP was largely symbolic:

- grammars and dictionaries were hand-crafted,
- translation rules were explicitly specified,
- systems were interpretable but brittle.

One influential framing was the noisy-channel view of translation:

$$\hat{e} = \arg \max_e P(e \mid f) = \arg \max_e P(f \mid e)P(e).$$

Source: Wikipedia: Machine Translation

Why Rule-Based Systems Hit Limits

Rule-based systems were attractive because failures could be inspected directly.

But they suffered from three major weaknesses:

- enormous manual effort,
- accumulation of exceptions and edge cases,
- brittleness outside narrow domains.

For open-ended language tasks, hand-written rules did not scale.

Statistical NLP

As larger text corpora became available, NLP moved toward statistical estimation.

A classical example is the n -gram model:

$$P(w_1, \dots, w_T) \approx \prod_{t=1}^T P(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

- major progress beyond rules,
- still short-context,
- weak semantic representation.

Neural Sequence Models Before Transformers

RNNs, LSTMs, and sequence-to-sequence models introduced continuous representations and variable-length processing.

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$$

However, two persistent problems remained:

- long-context modeling was difficult,
- recurrence limited parallel training efficiency.

A Compact Timeline of NLP Eras

Era	Representative methods	Main limitation
1950s–1970s	Rule-based translation, symbolic parsing, formal grammars	Too much expert labor
1980s–2000s	N-grams, HMMs, statistical MT	Short context
1990s–2010s	RNNs, LSTMs, seq2seq + attention	Scaling remained difficult
2017–today	Transformers, GPT-style pretraining	Reliability and compute

Historical Lesson

NLP was blocked not by lack of interest, but by inadequate representation and inadequate scale.

Several Ideas Reinforced One Another

Modern LLMs did not emerge from a single breakthrough. Their success came from several developments acting together:

- the Transformer architecture,
- large-scale generative pretraining,
- massive datasets and GPU clusters,
- improved post-training and alignment,
- efficient scaling strategies such as MoE.

What Self-Attention Changes

The Transformer replaced recurrence with self-attention.

Attention Question

When interpreting the current token, which other tokens in the context matter most?

For input matrix $\mathbf{X}$, we compute:

$$\mathbf{Q} = \mathbf{XW}^Q, \quad \mathbf{K} = \mathbf{XW}^K, \quad \mathbf{V} = \mathbf{XW}^V.$$

Source: Vaswani et al. (2017): Attention Is All You Need

Visualizing Self-Attention

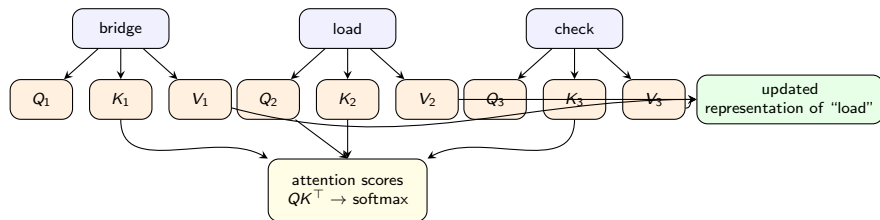


Figure: Self-attention: the token "load" compares against the full context instead of relying only on sequential memory.

Why the Self-Attention Figure Matters

The figure is useful pedagogically because it shows that:

- every token produces query, key, and value vectors,
- attention weights depend on relationships across the whole sequence,
- contextual meaning emerges from weighted mixing, not from a fixed lookup table.

This is why transformers scale better than recurrent sequence models for large language tasks.

GPT-Style Generative Pretraining

GPT-style models use a simple objective: predict the next token from the tokens that came before it.

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

or equivalently minimize:

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}).$$

Source: Brown et al. (2020): Language Models are Few-Shot Learners

Why This Objective Is So Powerful

The next-token objective looks modest, but at scale it forces the model to learn:

- syntax,
- semantics,
- style,
- factual associations,
- and latent task patterns.

Generative Pretraining

Train once on massive corpora, then adapt the resulting model through prompting, instruction tuning, or downstream tooling.

What Does “7B” or “70B” Mean?

In public discussion, model size is often summarized by parameter count.

- 7B → roughly 7 billion trainable weights,
- 70B → roughly 70 billion,
- 1T → roughly one trillion.

Example	Parameter count	Shorthand
Small open model	7×10^9	7B
Larger open model	70×10^9	70B
GPT-3 class	175×10^9	175B
Frontier-scale model	$> 10^{12}$	>1T

Parameter Count Is Not the Whole Story

Students should interpret parameter count carefully.

Do Not Oversimplify

Parameter count is not identical to intelligence, and it is not the same as memory in an ordinary human sense.

It is better to interpret it as a rough proxy for:

- representational capacity,
- engineering cost,
- deployment complexity.

Why Training Requires GPU Clusters

Training an LLM is far more demanding than running one for inference.

The system must repeatedly perform:

- forward passes,
- backpropagation,
- optimizer updates,
- large-scale tensor movement.

This is why modern LLM development is also a systems problem involving memory, networking, storage, and parallel computing.

Mixture of Experts (MoE)

As models grew, researchers explored sparse architectures such as mixture-of-experts.

$$y = \sum_{i=1}^M g_i(x) E_i(x)$$

- E_i denotes expert i ,
- $g_i(x)$ is a routing or gating score,
- only part of the network activates for a given token.

Source: Fedus et al. (2021): Switch Transformers

Training Is More Than Pretraining

Students often hear that LLMs are trained by “predicting the next token.” That is true, but incomplete.

Modern systems usually go through several stages:

- pretraining,
- continued or domain-adaptive training,
- supervised fine-tuning,
- preference or reinforcement-based post-training.

Fine-Tuning, RLHF, and DPO

Once a base model exists, several customization strategies are possible.

- **SFT:** learn from demonstrations,
- **RLHF:** learn from reward signals based on feedback,
- **DPO:** learn directly from preferred vs. dispreferred response pairs,
- **LoRA-like methods:** update only small adapter-style components.

The model that users interact with is usually a post-trained assistant, not a raw pretrained model.

Decoder-Only Transformer Pipeline

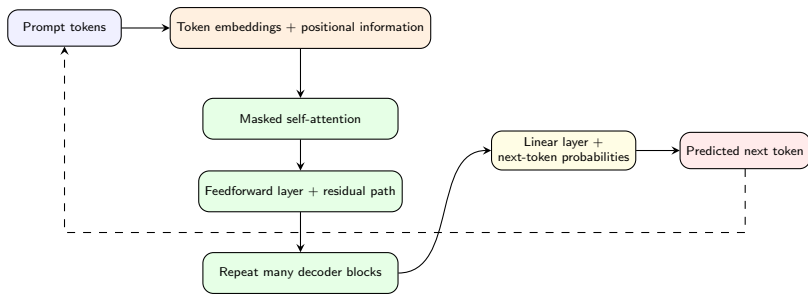


Figure: Decoder-only transformer architecture typical of GPT-style LLMs.

Why This Architecture Figure Matters

Many famous transformer diagrams were created for translation and show encoder-decoder structure.

Important Distinction

The GPT family that dominates modern text generation is usually decoder-only and autoregressive.

That distinction matters when students later study prompting, reasoning models, and coding assistants.

Computers Need Numerical Representations

To understand why LLMs are useful, we need one conceptual shift:

Core Shift

Computers do not process words directly as meanings. They process numerical representations.

The first stage of the pipeline is therefore not “understanding” but representation.

Tokenization Basics

Before text can be processed, it is split into **tokens**.

A token may be:

- a whole word,
- part of a word,
- punctuation,
- a short character sequence.

Example:

`"Design loads must be checked carefully."`

may become

`[Design, loads, must, be, checked, carefully, .].`

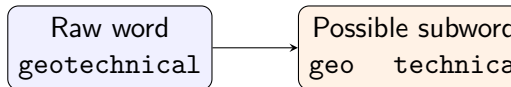
Byte-Pair Encoding (BPE) Intuition

A popular tokenizer family is based on subword merges.

Step	Frequent pair	Merge result
1	d + esign	design
2	load + s	loads
3	deflect + ion	deflection
4	reinforce + ment	reinforcement

Source: Hugging Face: Tokenizer Summary

Visualizing Subword Tokenization



Common engineering words may remain compact, while rare technical words are broken into subwords.

Figure: Subword tokenization of a technical term.

Why Tokenization Matters in Engineering

A phrase like “ACI 318 load combination” contains:

- ordinary English,
- a code identifier,
- domain-specific technical terminology.

Practical Consequence

Tokenization affects cost, context-window usage, and sometimes output quality, especially in code-heavy or standards-heavy workflows.

What an Embedding Is

Each token is mapped to a vector of real numbers called an **embedding**.

If vocabulary size is $|\mathcal{V}|$ and embedding dimension is d , then the embedding table is:

$$E \in \mathbb{R}^{|\mathcal{V}| \times d}.$$

Looking up a token returns one row of this matrix.

Source: Wikipedia: Word Embedding

Token versus Embedding

Definition

A token is a discrete symbol. An embedding is its continuous numerical representation.

Tokenization discretizes language.

Embeddings place that language in a vector space where similarity can be computed.

Why High-Dimensional Spaces Matter

Embeddings often live in hundreds or thousands of dimensions. This makes it possible to encode many weak signals simultaneously:

- syntax,
- topic,
- domain,
- tone,
- contextual association.

Cosine similarity is a common comparison measure:

$$\text{cos_sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

Embedding Space as Geometry

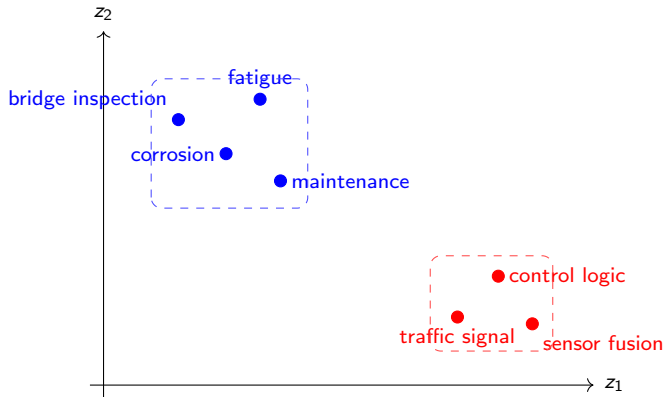


Figure: Low-dimensional view of embedding-space clustering.

Why the Geometry Is Useful

The point of the figure is not that embeddings are really two-dimensional.

Geometric Intuition

Semantically related items tend to lie near one another in the embedding space, even when the wording is not identical.

That is why a query about “steel design loads” can retrieve passages phrased as “load combinations for structural steel members.”

Front-End Representation Pipeline

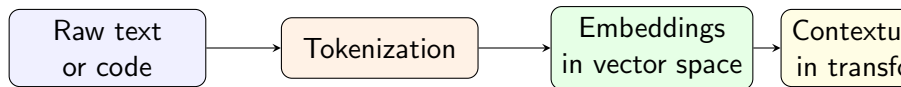


Figure: Front-end representation pipeline for an LLM.

Topic 15 emphasizes the first half of this pipeline: how text becomes vectors.

Sentence and Document Embeddings

In engineering workflows, we often compare larger units than single tokens:

- sentences,
- paragraphs,
- code blocks,
- report sections,
- table descriptions.

Conceptually, one simple aggregation idea is:

$$\mathbf{s} = \frac{1}{m} \sum_{i=1}^m \mathbf{e}_i.$$

Chunking Choices for Engineering Documents

Source material	Possible chunk
Design code PDF	one clause or paragraph
Structural report	one subsection or executive-summary paragraph
Spreadsheet note export	one row with neighboring notes
Python script	one function or docstring-plus-function block
Inspection log	one observation entry

Chunking matters because fragments that are too small lose meaning, while chunks that are too large become noisy and expensive.

What a Vector Store Does

Once documents or code blocks are converted into embeddings, those vectors can be stored and searched.

Vector Store

A vector store is a database or indexing system designed to store numerical embeddings together with metadata and original text.

Basic Retrieval Pipeline

The basic pipeline is straightforward:

- ① split documents into chunks,
- ② embed each chunk as a vector,
- ③ store vector + metadata + original text,
- ④ embed the user query,
- ⑤ retrieve nearby vectors,
- ⑥ return the associated human-readable chunks.

Source: Lewis et al. (2020): Retrieval-Augmented Generation

Similarity Search

Suppose a query embedding is $\mathbf{q}$ and stored chunk embeddings are $\mathbf{v}_1, \dots, \mathbf{v}_N$.

A simple retrieval score is:

$$s_i = \text{cos_sim}(\mathbf{q}, \mathbf{v}_i)$$

and the system returns the top- k chunks:

$$\mathcal{R}_k(\mathbf{q}) = \text{TopK}_{i=1, \dots, N} s_i.$$

In production, systems often use approximate nearest-neighbor indexing, reranking, or hybrid retrieval.

Keyword Search versus Semantic Search

Query type		Keyword search strong when	Semantic search strong when
Exact lookup	clause	exact known	citation wording varies
Prior search	project	terminology repeated	re-ideas paraphrased
Spreadsheet lookup		labels exact	assumptions in prose
Research report	sup-	proper nouns matter	concept similarity matters

In engineering, hybrid retrieval is often the natural choice.

How Does a Sentence Come Back?

A vector store does not store meaning in ordinary words alone. Each stored item looks conceptually like:

ID	Vector	Source metadata	Original text
17	$\mathbf{v}_{17}$	Report A, p. 42	"Deflection under service load sha
18	$\mathbf{v}_{18}$	Report B, p. 11	"Load combinations for steel beam

The vector is the search key; the original text remains the payload shown to the user.

Nearest-Neighbor Retrieval as Geometry

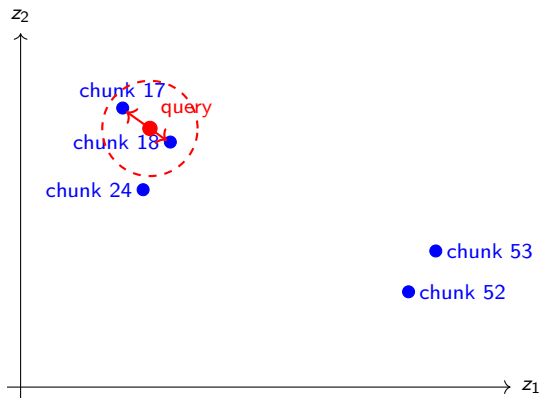


Figure: Nearest-neighbor retrieval in an embedding space.

Why the Retrieval Figure Matters

The retrieval picture shows three key ideas:

- the query is embedded into the same vector space as stored chunks,
- retrieval returns nearby chunk IDs rather than “meaning” directly,
- metadata and original text are required to recover usable engineering evidence.

Source: FAISS: similarity search and clustering

A Small Numerical Example

Suppose the query is

$q = \text{"serviceability limit for beam deflection"}$

and the similarities are:

- “Deflection under service loads shall remain within specified limits.” $\rightarrow 0.92$
- “Ultimate strength combinations control flexural resistance.” $\rightarrow 0.67$
- “Retaining wall drainage layers reduce pore pressure.” $\rightarrow 0.18$

The top result is semantically relevant even if the exact phrase “beam deflection” is absent.

Why Metadata Matters

Metadata is the bridge back to the engineering source of truth. Engineers usually need to know:

- which document the chunk came from,
- what page or section it belongs to,
- whether the source is current, approved, or superseded,
- whether it came from a code, report, note, or codebase.

Without metadata, semantic similarity alone does not support traceability or auditability.

What Hallucination Means

In this context, a hallucination is output that is:

- fluent,
- plausible,
- but unsupported, incorrect, fabricated, or overconfident.

Engineering Interpretation

An LLM is optimized to continue text plausibly, not to guarantee truth in the strict engineering sense.

Source: Wikipedia: Hallucination (artificial intelligence)

Why Hallucinations Occur

Several mechanisms contribute:

- probabilistic next-token generation,
- incomplete context,
- post-training pressure to be helpful,
- distribution shift on specialized technical questions,
- decoding choices.

This explains why a model may sound confident when it should instead say, “I need the source document.”

Temperature and Output Reliability

Temperature scaling changes how aggressively the model explores alternatives:

$$P_T(v) \propto \exp\left(\frac{\log P(v)}{T}\right)$$

Temperature	Behavior	Typical use
Low (0–0.3)	focused, low variance	coding, factual QA
Medium (0.4–0.7)	balanced	analysis, summarization
High (0.8+)	exploratory	brainstorming

Lower Temperature Is Not a Cure

Important Limitation

Lower temperature may reduce some variation, but it does not solve missing knowledge, weak retrieval, or poor calibration. A deterministic wrong answer is still wrong.

This is why reliability must be designed into the workflow, not delegated to one decoding setting.

How Engineers Should Respond

In engineering settings, hallucination risk should be reduced by:

- grounding answers in retrieved documents or cited standards,
- using lower-variance decoding for factual tasks,
- asking for assumptions, sources, and intermediate checks,
- treating outputs as drafts rather than final truth,
- keeping a human reviewer in the loop.

Putting the Pieces Together

The document-retrieval pipeline that connects LLMs to engineering knowledge work is:

- ① split technical documents into chunks,
- ② convert each chunk into an embedding,
- ③ store vectors and metadata in a searchable index,
- ④ embed the query into the same space,
- ⑤ retrieve nearby vectors,
- ⑥ return the original text chunks for synthesis or review.

Chapter Takeaways

- LLMs matter in engineering because engineering work is document-heavy.
- NLP historically struggled with both representation and scale.
- Transformers, pretraining, and large-scale compute changed the field.
- Tokenization and embeddings are the bridge from text to computation.
- Vector retrieval makes meaning-based document search practical.
- Hallucination remains a serious limitation and requires human oversight.

From Topic 15 to Topic 16

Topic 15 focused on motivation and representation.

Topic 16 will build on this foundation by asking:

- how modern LLM systems are organized internally,
- how prompting and workflow design change engineering practice,
- how retrieval, knowledge engineering, and agentic systems extend the base model.

That discussion will naturally lead toward Topic 17 and human-in-the-loop engineering.